Python-Basics

immutable: int float bool str tuple · mutable: list dict set · falsy: 0 0.0 '' [] {} set() None

`-7 // 2` -4 (rundet ab) · `7 % 3` → 1 · `2**3` → 8 · `1/2` → 0.5

`0 or 'x'` → `'x'` · `[1] and [2]` → `[2]` or/and geben Operanden zurück · is: selbes Objekt, ==: Inhalt · 0.1+0.2 != 0.3

`f'{x:.2f}' f'{s:<8}' f'{n:>5}' f'{n:05d}'` Format

`s.split(',') ','.join(L) s.strip() s.replace(a,b) s.find(x)` -1 = nicht gefunden

`L[start:stop:step]` stop exklusiv; L[::-1] umkehren

`L.append(x) L.pop() L.remove(x) L.insert(i,x) L.sort()` in-place → None

`sorted(L, key=len, reverse=True)` neue Liste

`b = a` KEINE Kopie! a[:] a.copy() list(a) copy.deepcopy(a)

`a = a + [8] / a += [8]` neues Objekt / in-place (b sieht's)

`d.get(k, 0) d.keys() d.values() d.items() k in d` in prüft Keys

`s1 | s2, s1 & s2, s1 - s2, s1 ^ s2` Set-Ops; {} = leeres dict!

`[x*2 for x in L if x > 0]` List Comprehension

`{k: v for k, v in d.items()} {x % 3 for x in L}` dict / set

`(x*x for x in L)` Generator: lazy, nur 1× iterierbar

`enumerate(L, start=1) zip(a, b)` zip: kürzeste gewinnt

`list(map(f, L)) list(filter(f, L)) lambda x: x*2`

`def f(a, b=2, *args, **kwargs)` args=tuple, kwargs=dict

`def f(L=[])` Falle! → L=None; if L is None: L = []

`a, *b, c = [1, 2, 3, 4]` b = [2, 3]

kein Block-Scope (if/for) · LEGB: Local → Enclosing → Global → Built-in

`try: … except ValueError as e: … else: … finally: …` else: ohne Fehler, finally: immer

`raise ValueError(f"ungültig: {x}")` class MyError(Exception): pass

TypeError 1+'a' · ValueError int('a') · IndexError · KeyError · NameError · ZeroDivisionError

`with open(p, 'r', encoding='utf-8') as f:` 'w' 'a' 'rb'; for line in f

`class C(Base): __init__(self) super().__init__()` __repr__ / __str__ (fehlt → __repr__) / __eq__ (+__hash__)

`@classmethod (cls) @staticmethod (-) self.__x` → `_C__x` Name Mangling

if __name__ == '__main__': · PEP 8: snake_case, UPPER_CASE, CapWords, 4 Leerz., 79 Zeichen

NumPy · Erzeugen

```
import numpy as np
```

`np.array([1, 2, 3]) np.array([[1,2], [3,4]])` 1 dtype! Up-Cast: [1, 2.5] → float64, [1,'2'] → str

`np.zeros(5, dtype=int) np.ones((2,3)) np.full((2,2), 7)`

`np.arange(0, 10, 2)` [0 2 4 6 8] stop exklusiv

`np.linspace(0, 1, 5)` [0 .25 .5 .75 1] Ende inklusiv

`np.empty(3) np.eye(2)` uninitialisiert / Einheitsmatrix · np.random.seed(0)

`np.random.random((3,3))` [0, 1)

`np.random.normal(0, 1, (2,3)) np.random.randn(5)`

`np.random.randint(0, 10, (3,3))` ganz, [0, 10)

`a.astype(float)` dtype ändern · rng = np.random.default_rng(0)

Standard float64/int64 · int8 −128..127 · uint8 0..255 (Überlauf!) · Bytes = Bits/8

NumPy · Attribute & Indexing

`a.ndim a.shape a.size a.dtype a.itemsize a.nbytes` (3,4,5): 3, (3,4,5), 60, 8, 480

`str(a.dtype)` 'float32'; np.arange(5).shape → (5,)

`M[1, 2] M[-1, -1]` [zeile, spalte]

`M[0] = M[0, :] · M[:, 0]` Zeile 0 / Spalte 0

`M[:2, :3] M[1:-1, 1:-1] M[::-1, ::-1]`

`b = M[:2]` VIEW → b[0]=9 ändert M!

`b = M[:2].copy()` Kopie; np.shares_memory(b, M)

`x[[1, 5, 8]]` Fancy Indexing → Kopie

`x[x > 5]` Boolean Indexing → Kopie

`x[(x > 2) & (x < 7)] x[(x < 2) | (x > 8)] x[~(x > 5)]` Klammern Pflicht! kein and/or/not

`x[x < 0] = 0` Werte per Maske setzen

`(x > 5).sum()` Anzahl True; np.any / np.all

NumPy · Form

`a.reshape(3, 4) a.reshape(3, -1)` -1 = automatisch; Größe muss passen

`a.reshape(1, -1) / a.reshape(-1, 1)` Zeilen- (1,n) / Spaltenvektor (n,1)

`np.concatenate([x, y]) np.hstack([x, y])` nebeneinander

`np.vstack([x, y])` untereinander → (2, n)

`np.split(x, 3) np.split(x, [2, 5])` 3 gleiche Teile / an Index 2 und 5

NumPy · Rechnen

`a + 10, a * 2, a ** 2, a + b, a / b, a // b, a % b` elementweise (ufuncs: np.add …)

`[1,2]*2` → `[1,2,1,2]` · `np.array([1,2])*2` → `[2 4]`

vektorisiert statt Schleife (~100–1000× schneller) · %timeit f()

Broadcasting: Formen von hinten vergleichen, gleich oder 1

`M * np.array([10, 100, 1000])` (2,3)*(3,) je Spalte

`M / M.sum(axis=1).reshape(-1, 1)` Zeilen normieren; ohne reshape: ValueError

`np.sum np.prod np.mean np.median np.std np.var np.min np.max np.percentile(a, 90)`

`np.argmin np.argmax` INDEX des Min/Max

`np.nansum np.nanmean np.nanmax …` NaN ignorieren (nicht any/all)

`np.sum([1, nan])` → `nan` · `np.isnan(a)` nan == nan → False!

`M.sum() M.sum(axis=0) M.sum(axis=1)` alles / je Spalte ↓ / je Zeile →

angegebene Achse verschwindet: (3,4) axis=0 → (4,), axis=1 → (3,)

`A @ B = A.dot(B)` (m,n)@(n,p) → (m,p); innere müssen passen

`A * B` elementweise! · A.T transponiert

`np.array_equal(a, b) np.allclose(a, b)` exakt / mit Toleranz (Floats)

`if a == b:` ValueError (Array, kein bool)

`np.save('x.npy', a) np.load('x.npy') np.savez('d.npz', a=a, b=b)`

pandas · Series

```
import pandas as pd
```

`s = pd.Series([1, 2], index=['a', 'b'], name='temp')`

`pd.Series({'a': 1, 'b': 2}) pd.Series(5, index=[1, 2, 3])` Dict-Keys → Index

`s.index s.values s.name s.index.name`

`s.rename('pop').rename_axis('year')` Name / Indexname

`a + b` über Index! fehlt in einem → NaN, Ergebnis float

`a.add(b, fill_value=0)` fehlende = 0

`s.loc['b'] s.loc['a':'c']` LABEL, Ende INKLUSIV

`s.iloc[0] s.iloc[1:3]` POSITION, Ende exklusiv

`s[s > 3] s[(s > 3) & (s < 9)]` Filtern

s[1] bei String-Index → KeyError (pandas 3): immer loc/iloc

`s.sort_values(ascending=False).head(3)`

`s.apply(f) df['x'].apply(f)` langsamer als vektorisiert; .round(1) .astype(int)

`s.isna() (=isnull) s.isna().sum() s.dropna() s.fillna(0) s.fillna(s.mean())`

`s.mean()` ignoriert NaN (≠ np.mean!) · len(s) vs s.count() (ohne NaN)

None → NaN, Spalte wird float64 · fillna(0) verfälscht mean

`s.tolist() s.to_dict() s.value_counts()`

pandas · DataFrame

`pd.DataFrame({'name': [...], 'note': [...]})` Dict von Listen

`pd.DataFrame({'Area': d1, 'Pop': d2})` Dict von Dicts: außen Spalten

`pd.DataFrame(arr, index=['a','c'], columns=['x','y'])`

`pd.read_csv('f.csv') df.to_csv('f.csv', index=False)` leer → NaN

read_/to_: csv json html xml excel parquet pickle sql

`df.head(3) df.tail() df.shape df.columns.tolist()`

`df.describe()` count mean std min 25% 50% 75% max

`df.info()` Spalten, dtypes, non-null, Speicher

`df['col']` Series · df[['a','b']] ¦ DataFrame

`df['zeile']` KeyError! [] = Spalten → df.loc['zeile']

`df.loc['c', 'Ohio'] df.loc[:, 'Ohio'] df.loc['a':'c', ['x', 'y']]`

`df.iloc[0] df.iloc[:2, 0] df.iloc[1, 2]`

`df[df['note'] < 2.0]` Zeilen filtern

`df[(df.a > 1) & (df.b == 'x')]`

`df.loc[df['x'] > 2, 'y']` filtern + Spalte → Series

`df['dichte'] = df['pop'] / df['flaeche']` neue Spalte

`r = df.copy(); r['ok'] = r['note'] <= 4.0` Original unverändert

`df = df.sort_values(by='x', ascending=False)` gibt NEUEN df zurück!

`df.reset_index(drop=True)` Index 0, 1, 2 …

`df.rename(columns={'a': 'b'}) df.drop(columns=['a']) df.dropna() df.isna().sum()`

`for i, row in df.iterrows(): row['x']` zeilenweise (langsam)

pandas · Join, Concat, Group-by

`pd.merge(df1, df2, on='key')` inner (Standard): nur Keys in beiden

`how='left' / 'right' / 'outer'` alle links / rechts / alle; fehlt → NaN

Key mehrfach → jede Kombination (3×a links, 1×a rechts → 3 Zeilen)

`pd.concat([s1, s2]) pd.concat([df1, df2])` aneinanderhängen

split → apply → combine

`df.groupby('k')['x'].mean()` Series je Gruppe

`df.groupby('k')['x'].agg(['mean', 'sum', 'count'])`

`df.groupby(['k1', 'k2'])['x'].sum() df.groupby('k').size()`

`.rename(columns={'mean': 'my_mean'})`

Web · requests

```
import requests; from io import StringIO
```

`H = {"User-Agent": "Mozilla/5.0 (DataPy, BHT)"}`

`r = requests.get(url, headers=H, timeout=10)`

`r.status_code r.text r.content r.json() r.raise_for_status()`

1xx Info · 2xx OK (200) · 3xx Umleitung · 4xx Client (404, 403) · 5xx Server (500) · 404 löst keine Exception aus (nur raise_for_status)

`tables = pd.read_html(StringIO(r.text), match='Pop', header=0)` LISTE!

`df = tables[0]; df = df[['Year', 'Pop.']]` bereinigen

erst API/Download prüfen · REST: GET url?param=x → JSON

`d['data']['city']['name']` verschachteltes JSON

`if d['status'] != 'ok': return None`

Recht: §87a UrhG schützt Investition, keine wesentl. Teile · §60d TDM Forschung · robots.txt, AGB, DSGVO · Crawling: Scrapy, Pausen

BeautifulSoup

`from bs4 import BeautifulSoup`

`soup = BeautifulSoup(html, 'html.parser')` oder r.content

`soup.title.get_text() soup.find('h1').text`

`soup.find('p')` ERSTES Element oder None

`soup.find_all('p')` LISTE (ResultSet) oder []

`soup.find_all('div', class_='x')` class_ mit _ !

`soup.find_all(id='main') soup.find('a', href=True)`

`tag.get_text().strip()` Text ohne Tags/Rand-Leerraum

`a.get('href') / a['href']` fehlt: None / KeyError

`div.find('span', class_='preis')` nur INNERHALB von div

`[c.name for c in soup.children]` [None, 'html', None]

`soup.find('h2').text` kein h2 → AttributeError (None)

find_all(…).text geht nicht → for-Schleife/Comprehension

`[(a.get_text().strip(), a.get('href')) for a in soup.find_all('a') if a.get('href')]`

`pd.DataFrame(rows)` Liste von Dicts → DataFrame

`float('2,90 €'.replace('€', '').strip().replace(',', '.'))` 2.9